

Instrumentation and Measurements Lab

Digital Bogy Temperature Monitoring System

Project By: Abdul Basit Butt

1. Introduction

Temperature monitoring plays a critical role in modern engineering applications, ranging from environmental monitoring and industrial process control to healthcare and safety systems. With the increasing availability of low-cost microcontrollers and digital sensors, accurate and reliable temperature measurement systems can now be developed using compact embedded platforms.

This project focuses on the design and implementation of a **digital body temperature monitoring system** using an **Arduino microcontroller** and a **DS18B20 digital temperature sensor**. Unlike traditional analog temperature sensors, digital sensors provide factory-calibrated output, improved noise immunity, and reliable communication over digital protocols. The system is capable of displaying real-time temperature values on an LCD and triggering an audible alarm when the temperature exceeds a predefined threshold.

The project emphasizes **correct sensor selection**, **robust signal processing**, and **clear distinction between measured surface temperature and estimated body temperature**, ensuring engineering correctness rather than superficial readings. However, achieving stable and accurate temperature measurement using low-cost sensors requires careful sensor selection, signal conditioning, and system design, which this project aims to address.

2. Project Objectives

The primary objectives of this project are:

1. To design a reliable temperature monitoring system using a digital temperature sensor
2. To measure and display real-time temperature values accurately
3. To estimate body temperature based on calibrated sensor readings
4. To implement an audible alert system using a buzzer for abnormal temperature detection
5. To demonstrate proper embedded system design practices, including sensor interfacing, data processing, and user feedback

3. Project Overview

The system consists of an Arduino-based embedded controller interfaced with a DS18B20 digital temperature sensor. The sensor measures temperature and communicates digitally with the microcontroller using the One-Wire protocol. The measured temperature is processed, filtered, and displayed on a 16x2 LCD.

To enhance usability and safety, a buzzer is integrated into the system. The buzzer is activated when the measured temperature exceeds a predefined threshold, indicating a possible fever or abnormal thermal condition.

The system is designed to be modular, scalable, and robust, making it suitable for educational demonstrations as well as real-world monitoring applications.

4. Components Used and Their Purpose

I. Arduino UNO

The Arduino UNO serves as the **central control unit** of the system. It is responsible for:

- Communicating with the temperature sensor
- Processing and filtering temperature data
- Controlling the LCD display
- Activating the buzzer based on alarm conditions

II. DS18B20 Digital Temperature Sensor

The DS18B20 is a **digital temperature sensor** that provides accurate and stable temperature readings over a One-Wire communication interface.

Purpose in the project:

- Measures temperature digitally, eliminating analog noise and ADC errors
- Provides factory-calibrated output for improved accuracy
- Enables reliable communication using a single data pin

III. 16x2 LCD Display

The 16x2 Liquid Crystal Display is used to present real-time temperature data in a human-readable format.

Purpose in the project:

- Displays measured temperature values
- Provides immediate visual feedback to the user
- Enhances system usability without requiring external devices

IV. Buzzer

The buzzer acts as an **audible alert mechanism**.

Purpose in the project:

- Activates when temperature exceeds a predefined threshold

- Provides immediate warning without visual attention
- Enhances safety and responsiveness of the system

V. 4.7k Ω Pull-Up Resistor

The resistor is required by the One-Wire protocol to ensure reliable data transmission.

Purpose in the project:

- Maintains the data line in a stable HIGH state
- Ensures reliable digital communication between Arduino and DS18B20
- Prevents communication errors and sensor disconnection issues

VI. Potentiometer (10 k Ω)

The potentiometer is used to adjust the LCD contrast.

Purpose in the project:

- Allows clear visibility of displayed characters
- Ensures readability under varying lighting conditions

VII. Breadboard and Jumper Wires

These components are used for prototyping and circuit assembly.

Purpose in the project:

- Enables rapid circuit construction without soldering
- Allows easy debugging and modification
- Provides flexible connection management

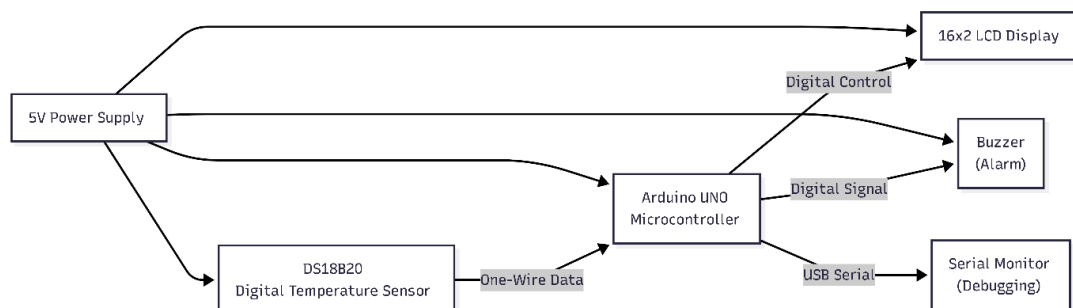
5. System Architecture

I. Hardware Interfacing and Pin Configuration

This table presents the complete hardware interfacing details of the digital body temperature monitoring system. It specifies the connections between the Arduino UNO and each peripheral component, ensuring correct signal routing, power distribution, and system functionality.

Component	Pin / Wire	Connected To (Arduino UNO)	Purpose
DS18B20 Temperature Sensor	VCC	5V	Power supply for sensor
	GND	GND	Common ground
	DATA	Digital Pin D7	One-Wire temperature data
	4.7 kΩ Resistor	Between DATA and 5V	One-Wire pull-up resistor
16x2 LCD Display	Pin 1 (VSS)	GND	LCD ground
	Pin 2 (VDD)	5V	LCD power supply
	Pin 3 (VO)	Potentiometer (middle pin)	LCD contrast control
	Pin 4 (RS)	Digital Pin D12	Register select
	Pin 5 (RW)	GND	Write mode selection
	Pin 6 (E)	Digital Pin D11	Enable signal
	Pin 11 (D4)	Digital Pin D5	LCD data bit
	Pin 12 (D5)	Digital Pin D4	LCD data bit
	Pin 13 (D6)	Digital Pin D3	LCD data bit
	Pin 14 (D7)	Digital Pin D2	LCD data bit

Component	Pin / Wire	Connected To (Arduino UNO)	Purpose
	Pin 15 (A)	5V	LCD backlight anode
	Pin 16 (K)	GND	LCD backlight cathode
Potentiometer (10 k Ω)	One end	5V	Contrast voltage reference
	Other end	GND	Contrast voltage reference
	Middle pin	LCD Pin 3 (VO)	LCD contrast adjustment
Buzzer	Positive (+)	Digital Pin D8	Audible alarm signal
	Negative (-)	GND	Buzzer ground
Arduino UNO	USB Port	Computer	Programming and Serial monitoring



The block diagram illustrates the overall system architecture of the digital body temperature monitoring system. The DS18B20 digital temperature sensor measures

temperature and communicates with the Arduino UNO using the One-Wire protocol. The Arduino processes the temperature data and displays it on a 16x2 LCD. When the temperature exceeds a predefined threshold, the Arduino activates a buzzer to provide an audible alert. Serial communication is used for debugging and monitoring purposes. All components are powered by a common 5V supply.

6. Working Principle

The DS18B20 sensor measures temperature and transmits the digital data to the Arduino via the One-Wire protocol. The Arduino processes this data, applies smoothing and calibration algorithms, and displays the temperature on the LCD.

If the measured temperature crosses the preset threshold, the Arduino activates the buzzer to alert the user. Serial communication is also used for debugging and verification during development. The system continuously monitors temperature values and evaluates them against predefined thresholds to determine alarm conditions.

I. Advantages of the System

- High accuracy due to digital sensing
- Reduced noise and improved reliability
- Modular and expandable design
- Clear visual and audible feedback
- Suitable for both educational and practical applications
- Factory-calibrated digital sensing reduces the need for complex analog calibration

7. System Code

This code is provided for reference and implementation verification.

/*

DS18B20 Body Temperature Monitor

- DS18B20 data pin -> D7 (with 4.7k pull-up to 5V)

- 16x2 LCD: LiquidCrystal lcd(12,11,5,4,3,2)

- Buzzer: D8 -> +, GND -> -

Features:

- Robust sensor error handling
- Exponential smoothing for stability
- Calibration offset (BODY_OFFSET) to estimate body/core temp from skin
- Alarm with hysteresis
- Runtime Serial commands:

"OFFSET <value>" -> set body offset (°C)

"ALARM <value>" -> set alarm ON threshold (°C)

"HYST <value>" -> set hysteresis (°C)

"STATUS" -> print current config

*/

```
#include <OneWire.h>
```

```
#include <DallasTemperature.h>
```

```
#include <LiquidCrystal.h>
```

```
// Hardware pins
```

```
#define ONE_WIRE_PIN 7
```

```
#define BUZZER_PIN 8
```

```
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);
```

```
OneWire oneWire(ONE_WIRE_PIN);
```

```
DallasTemperature sensors(&oneWire);
```

```
// Behavior/configuration (tweak as needed)
```

```
float bodyOffset = 3.0;    // estimated core = skin + offset (calibrate this)
```

```
float alarmOn = 38.0;     // alarm ON threshold (°C)
```

```
float hysteresis = 0.5;    // hysteresis amount (°C)
```

```
const float SMOOTH_ALPHA = 0.25; // EMA smoothing factor (0..1) — larger =  
faster response
```



```
const unsigned long MEASURE_INTERVAL = 1000UL; // ms
```

```
// Runtime state
```

```
bool alarmState = false;
```

```
float smoothedTemp = NAN;
```

```
unsigned long lastMeasureMillis = 0;
```

```
void setup() {
```

```
    Serial.begin(9600);
```

```
    sensors.begin();
```

```
    pinMode(BUZZER_PIN, OUTPUT);
```

```
    digitalWrite(BUZZER_PIN, LOW);
```

```
    lcd.begin(16, 2);
```

```
    lcd.clear();
```

```
    lcd.print("Body Thermo Init");
```

```
    delay(500);
```

```
    Serial.println("DS18B20 Body Temp Monitor started.");
```

```
    printStatus();
```

```
}
```

```
void loop() {
```

```
    // Handle serial commands if present
```

```
    handleSerial();
```

```
    unsigned long now = millis();
```

```
    if (now - lastMeasureMillis >= MEASURE_INTERVAL) {
```

```

    lastMeasureMillis = now;
    measureAndUpdate();
}

// Optionally: do other non-blocking tasks here
}

void measureAndUpdate() {
    // Request temperature
    sensors.requestTemperatures(); // synchronous, but quick for one device
    float rawTemp = sensors.getTempCByIndex(0);

    // Check for disconnected sensor sentinel
    const float DISCONNECTED = DEVICE_DISCONNECTED_C; // typically -127
    if (rawTemp == DISCONNECTED || isnan(rawTemp)) {
        // Sensor error state
        Serial.println("ERROR: DS18B20 sensor not found or disconnected.");
        lcd.clear();
        lcd.setCursor(0,0);
        lcd.print("Sensor Error");
        lcd.setCursor(0,1);
        lcd.print("Check wiring");
        digitalWrite(BUZZER_PIN, LOW);
        return;
    }

    // Apply exponential smoothing to raw sensor (smooths noise but keeps
    responsiveness)
    if (isnan(smoothedTemp)) {

```

```

    smoothedTemp = rawTemp; // initialize on first good reading
} else {
    smoothedTemp = smoothedTemp + SMOOTH_ALPHA * (rawTemp -
smoothedTemp);
}

// Compute estimated body temp
float estBody = smoothedTemp + bodyOffset;

// Alarm logic with hysteresis
if (!alarmState && estBody >= alarmOn) {
    alarmState = true;
    digitalWrite(BUZZER_PIN, HIGH);
} else if (alarmState && estBody <= (alarmOn - hysteresis)) {
    alarmState = false;
    digitalWrite(BUZZER_PIN, LOW);
}

// Update LCD (clean formatting)
lcd.clear();
lcd.setCursor(0,0);
lcd.print("Skin: ");
lcd.print(smoothedTemp, 2);
lcd.print("C");

lcd.setCursor(0,1);
lcd.print("Body: ");
lcd.print(estBody, 2);
lcd.print("C ");

```

```

    lcd.print(alarmState ? "ALARM" : "OK");

    // Print to Serial for logging/debug
    Serial.print("RAW: "); Serial.print(rawTemp, 3);
    Serial.print(" C SM: "); Serial.print(smoothedTemp, 3);
    Serial.print(" C EST: "); Serial.print(estBody, 2);
    Serial.print(" C Alarm: "); Serial.println(alarmState ? "ON" : "OFF");
}

// ----- SERIAL COMMANDS -----
void handleSerial() {
    // Read one full line if available
    if (Serial.available() == 0) return;

    String line = Serial.readStringUntil('\n');
    line.trim();
    if (line.length() == 0) return;

    // Tokenize
    line.toUpperCase();
    if (line.startsWith("OFFSET ")) {
        float v = line.substring(7).toFloat();
        bodyOffset = v;
        Serial.print("BODY OFFSET set to: "); Serial.println(bodyOffset, 3);
        printStatus();
    }
    else if (line.startsWith("ALARM ")) {
        float v = line.substring(6).toFloat();
        alarmOn = v;
    }
}

```

```

    Serial.print("ALARM ON threshold set to: "); Serial.println(alarmOn, 2);
    printStatus();
}
else if (line.startsWith("HYST ")) {
    float v = line.substring(5).toFloat();
    hysteresis = v;
    Serial.print("HYST set to: "); Serial.println(hysteresis, 2);
    printStatus();
}
else if (line == "STATUS") {
    printStatus();
}
else if (line == "HELP") {
    Serial.println("Commands:");
    Serial.println(" OFFSET <value> - set body offset in °C");
    Serial.println(" ALARM <value> - set alarm ON threshold in °C");
    Serial.println(" HYST <value> - set hysteresis in °C");
    Serial.println(" STATUS      - print configuration");
}
else {
    Serial.print("Unknown command: "); Serial.println(line);
    Serial.println("Type HELP for commands.");
}
}

```

```

void printStatus() {
    Serial.println("---- Current Config ----");
    Serial.print("BODY OFFSET: "); Serial.println(bodyOffset, 3);
    Serial.print("ALARM ON:   "); Serial.println(alarmOn, 2);
}

```

```
Serial.print("HYSTERESIS: "); Serial.println(hysteresis, 2);  
Serial.print("SMOOTH ALPHA: "); Serial.println(SMOOTH_ALPHA, 3);  
Serial.println("-----");  
}
```

8. Conclusion

This project successfully demonstrates the design and implementation of a digital temperature monitoring system using Arduino and a DS18B20 sensor. By selecting an appropriate digital sensor and applying proper data handling techniques, the system achieves stable and reliable temperature readings. The integration of an LCD display and buzzer enhances usability and safety, making the system suitable for real-world monitoring scenarios and academic learning.